

FOOTBALLIQ

Azure Data Engineering Portfolio Project

End-to-End Setup Manual

Based on latest official Microsoft / Databricks documentation

Dataset	Transfermarkt — Kaggle (12 CSV, 756 MB) https://www.kaggle.com/datasets/davidcariboo/player-scores
Files	appearances, clubs, games, players + 8 more
Azure services	ADLS Gen2, Databricks Premium, Key Vault, Monitor
Architecture	Medallion: Bronze → Silver → Gold
Orchestration	Databricks Workflows (built-in job scheduler)
Catalog	Unity Catalog — auto-enabled on new workspaces

Table of Contents

1. Prerequisites & Accounts

1.1 Azure Account

[Sign up for Azure free account](#)

Go to: portal.azure.com and sign up.

You get \$200 credit for 30 days + 12 months of free services.

A credit card is required for identity verification only — you will NOT be charged if you stay within free limits.

IMPORTANT: You need the Premium plan for Databricks to use Unity Catalog. Free trial covers this.

1.2 Kaggle Dataset Download

1. Go to: [kaggle.com](https://www.kaggle.com) and create a free account.
2. Search for: Football Data from Transfermarkt (by davidcariboo).
3. Click Download — you get a ZIP containing 12 CSV files (756 MB total).
4. Extract the ZIP to a folder on your computer, e.g.: C:\footballiq\data\ or ~/\footballiq/data/

CSV File	Size & Key Info
appearances.csv	148 MB — 1.88M rows — player stats per game
club_games.csv	Bridge table — club results per game
clubs.csv	Team dimension — squad size, league
competitions.csv	League dimension — name, country, type
countries.csv	Country reference table
game_events.csv	Goals, cards, substitutions per minute
game_lineups.csv	Starting 11 and subs per game
games.csv	Match fact — scores, date, season
national_teams.csv	Player to national team mapping
player_valuations.csv	Market value history per player
players.csv	Player dimension — dob, position, nationality
transfers.csv	Transfer records — fees, clubs, dates

1.3 Power BI Desktop (for final step only)

Download free from powerbi.microsoft.com/desktop — only needed for the last section.

2. Create the Azure Resource Group

A Resource Group is a logical container for all project resources. Everything for FootballIQ lives in one resource group so you can manage, monitor, and delete it all together.

STEP 2

Create Resource Group — Azure Portal

5 min

5. Open portal.azure.com and sign in.
6. In the top search bar, type Resource groups and click it.
7. Click + Create (top left blue button).
8. Fill in the form:

Field	Value to Enter
Subscription	Your Azure free subscription (auto-selected)
Resource group name	rg-footballliq-dev
Region	East US 2 (pick the one closest to you — keep ALL resources in the same region)

9. Click Review + create.
10. Click Create. Wait ~10 seconds for the green 'Deployment succeeded' notification.
11. Click Go to resource group. You will see an empty resource group — this is correct.

Naming convention used throughout this manual

rg = resource group prefix (Azure best practice)

adls = Azure Data Lake Storage

dbw = Databricks Workspace

kv = Key Vault

ac = Access Connector

All names follow: prefix-footballliq-dev

3. Create Azure Key Vault

Key Vault securely stores secrets (like your ADLS access key). Databricks reads them at runtime using `dbutils.secrets.get()` so you never hardcode credentials in notebooks.

STEP 3.1 Create the Key Vault

5 min

12. In the Azure Portal search bar, type Key vaults and click it.

13. Click + Create.

14. Fill in the Basics tab:

Field	Value
Subscription	Your subscription
Resource group	rg-footballiq-dev
Key vault name	kv-footballiq-dev (must be globally unique — add your initials if taken e.g. kv-footballiq-dev-abc)
Region	East US 2
Pricing tier	Standard

15. Click the Access configuration tab at the top.

16. Under Permission model, select Vault access policy.

CRITICAL — Must be Vault access policy, NOT Azure RBAC

The Azure Key Vault-backed secret scope in Databricks ONLY supports the Vault access policy permission model.

If you leave it on Azure role-based access control, you will get a PERMISSION_DENIED error when Databricks tries to read secrets.

Source: Microsoft Learn — Secret management for Azure Databricks (2025)

17. Click Review + create, then Create. Wait for deployment.

18. Click Go to resource.

STEP 3.2 Store the ADLS Access Key in Key Vault

5 min

Note: Complete Section 4 (create ADLS) first, then come back here.

19. Go to your Key Vault page in Azure Portal.

20. In the left sidebar, click Objects > Secrets.

21. Click + Generate/Import.

22. Fill in:

Field	Value
Upload options	Manual
Name	adls-footballiq-key

Secret value	Paste the ADLS access key you will copy in Section 4.4
--------------	--

23. Click Create. The secret is now stored securely.

STEP 3.3

Grant Databricks Service Principal Access to Key Vault

5 min

This step was missing from older documentation — it is required in 2025

Databricks uses a global Azure service principal to read secrets from Key Vault.

Its fixed App ID is: 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d (same for ALL Azure tenants worldwide).

Without this step, `dbutils.secrets.get()` will throw `PERMISSION_DENIED` even though the scope is created correctly.

24. On your Key Vault page, click Access policies in the left sidebar.

25. Click + Create.

26. Under Secret permissions, check: Get and List. Click Next.

27. In the Principal search box, type: 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d

28. Select AzureDatabricks from the search results. Click Next.

29. Click Next (skip Application tab), then Create.

30. You should now see AzureDatabricks listed under Access policies.

4. Create Azure Data Lake Storage Gen2 (Raw Landing Zone)

ADLS Gen2 is the raw landing zone where all 12 CSV files land. Databricks reads from here into the Bronze layer. This is the ONLY storage layer you manage directly — everything else is handled inside Databricks Unity Catalog.

STEP 4.1

Create the Storage Account

5 min

31. In the search bar, type Storage accounts and click it.

32. Click + Create.

33. Fill in the Basics tab:

Field	Value
Subscription	Your subscription
Resource group	rg-footballiq-dev
Storage account name	adlsfootballiqdev (all lowercase, no hyphens, max 24 chars)
Region	East US 2 (must match all other resources)
Performance	Standard
Redundancy	Locally-redundant storage (LRS) — cheapest option, fine for portfolio

34. Click the Advanced tab at the top of the form.

35. Scroll down to the Data Lake Storage Gen2 section.

36. Check the box: Enable hierarchical namespace.

CRITICAL — Enable this NOW, cannot be changed later

Hierarchical namespace is what makes this ADLS Gen2 instead of regular Blob Storage.

Unity Catalog External Locations and Databricks ABFS access both require this.

This setting CANNOT be enabled after the storage account is created — you must do it now.

37. Leave all other settings as default. Click Review + create, then Create.

38. Wait for deployment, then click Go to resource.

STEP 4.2 Create the Raw Container and 12 Subfolders

10 min

39. On your storage account page, click Containers in the left sidebar.

40. Click + Container. Name: raw. Public access level: Private (no anonymous access). Click Create.

41. Click on the raw container to open it.

42. Click + Add Directory to create a subfolder. Create one for each CSV file:

Folder Name	CSV File to Upload	File Size
appearances	appearances.csv	148 MB
club_games	club_games.csv	~10 MB
clubs	clubs.csv	< 1 MB

competitions	competitions.csv	< 1 MB
countries	countries.csv	< 1 MB
game_events	game_events.csv	~50 MB
game_lineups	game_lineups.csv	~30 MB
games	games.csv	~5 MB
national_teams	national_teams.csv	< 1 MB
player_valuations	player_valuations.csv	~10 MB
players	players.csv	~5 MB
transfers	transfers.csv	~30 MB

STEP 4.3 Upload the 12 CSV Files

15 min

43. Click into the appearances folder.
44. Click Upload at the top. Browse to your Kaggle folder and select appearances.csv. Click Upload.
45. Repeat for each of the remaining 11 folders — each gets its matching CSV file.

[Large file tip — use Azure Storage Explorer for reliability](#)

appearances.csv is 148 MB. The browser upload can time out on slow connections.

Download Azure Storage Explorer (free desktop app) from: aka.ms/azstorage

It handles large files much more reliably and shows a proper progress bar.

STEP 4.4 Copy the ADLS Access Key

2 min

46. On your storage account page, click Security + networking > Access keys in the left sidebar.
47. Click Show next to key1.
48. Copy the Key value (not the Connection string — just the key itself).
49. NOW go back to Section 3.2 and store this key in Key Vault.

5. Create Azure Databricks Workspace

STEP 5.1

Create the Workspace

5 min

50. In the Azure Portal search bar, type Azure Databricks and click it.

51. Click + Create.

52. Fill in:

Field	Value
Subscription	Your subscription
Resource group	rg-footballiq-dev
Workspace name	dbw-footballiq-dev
Region	East US 2 (must match ADLS region exactly)
Pricing tier	Premium — REQUIRED for Unity Catalog (not Standard, not Trial)

Must be Premium tier — this is non-negotiable for Unity Catalog

Unity Catalog requires the Premium plan. The Standard plan will not give you access to Unity Catalog features.

Azure free trial covers Premium Databricks — you will not be charged if within trial limits.

If you accidentally create a Standard workspace, you cannot upgrade it — you must recreate it.

53. Leave Networking and other tabs as default.

54. Click Review + create, then Create.

55. Wait 3–5 minutes for deployment (this is normal — Databricks provisions multiple resources).

56. Once deployed, click Go to resource, then click Launch Workspace button.

57. This opens the Databricks workspace in a new tab — sign in with your Azure credentials.

STEP 5.2

Verify Unity Catalog is Auto-Enabled

3 min

Because you are creating a NEW workspace in 2024/2025, Unity Catalog is enabled automatically. Verify this:

58. In your Databricks workspace, look at the left sidebar.

59. Click the Catalog icon (looks like a book/database icon).

60. You should see Catalog Explorer open with a list of catalogs.
61. You will see at minimum: hive_metastore and a catalog named after your workspace (e.g. dbwfootballiqdev).
62. If you see these — Unity Catalog is active. You do NOT need to do anything else to enable it.

If Catalog Explorer shows nothing or an error

This means Unity Catalog is not enabled (rare on new workspaces).

Go to: accounts.azuredatabricks.net (the Databricks Account Console).

Sign in with the same Azure account. Click Catalog > find your region > assign the metastore to your workspace.

Then refresh your workspace — Catalog Explorer should now show catalogs.

6. Set Up Compute — Serverless (Current Default)

Confirmed from live testing: as of mid-2026, new free-trial Azure Databricks workspaces (Premium tier, Unity Catalog auto-enabled) often provision with ONLY the SQL Warehouses tab visible under Compute in the sidebar — there is no All-purpose Compute tab and no 'Create compute' button for classic clusters at all. This is not a permissions problem on your account; it reflects how Databricks is rolling out serverless-first workspaces. The good news: you do not need a classic cluster for this project. Serverless compute handles everything in this manual.

Why this is actually fine for this project

Serverless compute is available by default in Unity Catalog-enabled workspaces and requires zero setup.

It auto-scales, starts in seconds, and bills only for actual usage — no manual cluster sizing needed.

Every notebook in Sections 10–12 (Bronze/Silver/Gold) runs unchanged on Serverless — the PySpark code is identical regardless of compute type.

The only thing Serverless cannot do is read ADLS via the legacy `dbutils.fs.mount()` + Spark config approach — which is why this manual already uses External Locations (Section 8), the modern method

that works on both Serverless and classic clusters.

STEP 6.1 Attach a Notebook to Serverless Compute

3 min

63. In the Databricks sidebar, click + New > Notebook.
64. Name it: test_serverless. Default language: Python.
65. In the notebook toolbar (top-right area), click the Connect dropdown.
66. Select Serverless from the list. No configuration form appears — it attaches immediately.
67. Run a test cell:

```
print(spark.version)
# Should print something like: 3.5.x — confirms Spark session is live on Serverless
```

That is the entire setup. Use Serverless for every notebook in this manual — Sections 10 (Bronze), 11 (Silver), 12 (Gold), and the Workflow tasks in Section 13.

Cost note for Serverless

Serverless bills per-second of actual compute use, with idle notebooks auto-disconnecting after a period of inactivity (no charge while disconnected).

For a portfolio project processing 12 CSVs, expect well under \$5 in total compute charges — comfortably inside Azure free-trial credits.

There is no cluster to remember to terminate — one less thing to manage.

6.2 If Your Workspace DOES Show ‘Create compute’ (older or differently-provisioned workspaces)

Some workspaces — typically ones created earlier, on certain regions, or with specific admin configurations — still show the classic All-purpose Compute tab. If you see it, a classic cluster works identically for this project; use these settings:

Field	Value
Policy	Personal Compute
Cluster name	footballiq-cluster
Single node	Checked
Databricks Runtime	Latest LTS shown — e.g. 15.4 LTS (Spark 3.5)

Node type	Standard_DS3_v2 (default)
Terminate after	30 minutes (Advanced options > Auto termination)

If using a classic cluster, remember

Always terminate manually when done for the day — idle clusters still bill even at 30-min auto-terminate.

Wherever this manual says ‘attach to Serverless’, attach to footballiq-cluster instead — the notebook code does not change either way.

7. Connect Databricks to Key Vault (Secret Scope)

A Secret Scope lets Databricks read secrets from Key Vault at runtime. There is NO GUI button for this in the Databricks sidebar — you access it through a special URL. This is the official method from Microsoft Learn.

STEP 7.1

Get Key Vault URI and Resource ID

2 min

68. Go to your Key Vault (kv-footballiq-dev) in Azure Portal.

69. Click Settings > Properties in the left sidebar.

70. Copy and save these two values:

- Vault URI — looks like: `https://kv-footballiq-dev.vault.azure.net/`
- Resource ID — long string starting with `/subscriptions/xxxxxxxx-xxxx-.../resourceGroups/...`

STEP 7.2

Create the Key Vault-Backed Secret Scope

5 min

71. Go to your Databricks workspace tab in the browser.

72. Look at the URL in the address bar. It will look like:

`https://adb-1234567890123456.12.azuredatabricks.net/`

73. Modify the URL by adding `#secrets/createScope` at the end:

`https://adb-1234567890123456.12.azuredatabricks.net/#secrets/createScope`

Case-sensitive URL — the S in createScope MUST be uppercase

Correct: `#secrets/createScope` (capital S)

Wrong: #secrets/createscope (will show a 404 or blank page)

Source: Microsoft Learn — Secret management for Azure Databricks (May 2026)

74. Press Enter. A form page titled 'Create Secret Scope' appears.

75. Fill in:

Field	Value
Scope Name	footballiq-kv-scope
Manage Principal	All Users (for a personal portfolio project)
DNS Name	Paste your Key Vault URI from Step 7.1
Resource ID	Paste your Key Vault Resource ID from Step 7.1

76. Click Create. You should see a success message.

STEP 7.3

Test the Secret Scope in a Notebook

5 min

77. In the Databricks sidebar, click + New > Notebook.

78. Name it: test_secret_scope. Default language: Python. Attach to: Serverless (or footballiq-cluster if using a classic cluster).

79. In the first cell, type and run:

```
# Test that secret scope works
val = dbutils.secrets.get(scope='footballiq-kv-scope', key='adls-footballiq-key')
print('Secret retrieved:', val[:10]) # Shows first 10 chars as partial check
# Expected output: Secret retrieved: [REDACTED]
# Databricks intentionally hides the value — [REDACTED] means it WORKS
```

Troubleshooting — if you get PERMISSION_DENIED

Error 1: PERMISSION_DENIED on Key Vault

Fix: Go back to Section 3.3 and verify AzureDatabricks (App ID: 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d)

is listed under Key Vault > Access policies with Get and List permissions.

Error 2: Secret not found

Fix: Check the key name exactly matches 'adls-footballliq-key' (case-insensitive but check for typos).

Error 3: Scope not found

Fix: Run `dbutils.secrets.listScopes()` to list all scopes — verify the name matches.

8. Connect ADLS to Databricks via External Location (Modern Method)

Instead of the old `dbutils.fs.mount()` approach, Unity Catalog uses External Locations — a secure, governed way to connect ADLS that is registered in the catalog and auditable. This is the 2024/2025 standard.

STEP 8.1

Create an Access Connector for Azure Databricks

5 min

The Access Connector is a managed identity that gives Databricks secure access to ADLS without storing any credentials in code or configuration.

80. In Azure Portal search bar, type Access Connector for Azure Databricks and click it.

81. Click + Create.

82. Fill in:

Field	Value
Subscription	Your subscription
Resource group	rg-footballliq-dev
Name	ac-footballliq-dev
Region	East US 2

83. Click Review + create, then Create.

84. Once deployed, go to the resource. On the Overview page, copy the Resource ID.

STEP 8.2

Assign Storage Role to the Access Connector

3 min

85. Go to your ADLS storage account (adlsfootballiqdev) in Azure Portal.
86. Click Access Control (IAM) in the left sidebar.
87. Click + Add > Add role assignment.
88. Select Storage Blob Data Contributor. Click Next.
89. Under Assign access to, select Managed identity.
90. Click + Select members > change filter to Access connector for Azure Databricks > select ac-footballiq-dev.
91. Click Review + assign.

STEP 8.3 Create a Storage Credential in Databricks

5 min

92. In the Databricks workspace, click Catalog in the left sidebar.
93. Click the + Add button (top right of Catalog Explorer) > Add a storage credential.
94. Fill in:

Field	Value
Credential name	footballiq-adls-credential
Access Connector Resource ID	Paste the Resource ID from Step 8.1

95. Click Create.

STEP 8.4 Create an External Location

5 min

96. In Catalog Explorer, click + Add > Add an external location.
97. Fill in:

Field	Value
External location name	footballiq-raw-location
URL	abfss://raw@adlsfootballiqdev.dfs.core.windows.net/
Storage credential	footballiq-adls-credential

98. Click Create. Click Test connection — it should say Connection succeeded.

Understanding the ABFS URL format

abfss:// = Azure Blob File System Secure (requires Hierarchical Namespace = Gen2)

raw = your container name

@adlsfootballiqdev.dfs.core.windows.net = your storage account ADLS endpoint

/ = the root path (your 12 subfolders are inside here)

9. Set Up Unity Catalog — Create football_catalog

Unity Catalog is already enabled (confirmed in Section 5.2). Now you create your project catalog and the three medallion schemas inside it.

STEP 9.1 Create football_catalog

3 min

99. In the Databricks workspace, click Catalog in the left sidebar.

100. In Catalog Explorer, click + Add at the top > Add a catalog.

101. Fill in:

Field	Value
Catalog name	football_catalog
Type	Standard
Storage location	Leave blank (uses the metastore default — fine for portfolio)
Comment	FootballIQ Data Engineering Portfolio Project

102. Click Create. You will see football_catalog appear in the left panel.

Alternative: Create via SQL (faster if you prefer code)

Open a new notebook, attach to Serverless (or your cluster), and run:

```
CREATE CATALOG IF NOT EXISTS football_catalog COMMENT 'FootballIQ Portfolio Project';
```

Both methods work identically — the UI and SQL approach produce the same result.

Option A — Via SQL (recommended, faster for 3 schemas at once):

103. Open a new notebook. Attach to Serverless (or your cluster). Run all 3 commands:

```
-- Create the three medallion schemas
```

```
CREATE SCHEMA IF NOT EXISTS football_catalog.bronze COMMENT 'Raw ingested data, append-only, string types';
```

```
CREATE SCHEMA IF NOT EXISTS football_catalog.silver COMMENT 'Cleaned, typed, validated dims and facts';
```

```
CREATE SCHEMA IF NOT EXISTS football_catalog.gold COMMENT 'Business-ready aggregations and KPIs';
```

Github :

https://github.com/saizdevlab/de-project-footballiq-azure-resource-group/blob/Dev/00_setup/Catalog_Setup.ipynb

Option B — Via UI (one at a time):

104. In Catalog Explorer, expand football_catalog. Click + Add > Add a schema.

105. Set name to bronze. Click Create.

106. Repeat for silver and gold.

Verify the setup by running this in a notebook:

```
-- Verify your catalog structure
```

```
SHOW SCHEMAS IN football_catalog;
```

```
-- Expected output: bronze, silver, gold, (and possibly default)
```

Your full catalog structure is now:

football_catalog (your catalog)
├── bronze (schema)
├── silver (schema)
└── gold (schema)

10. Create Notebooks & Bronze Ingestion

STEP 10.1

Create the Notebook Folder Structure

5 min

107. In the Databricks sidebar, click Workspace (the grid icon).
108. Click on your username folder to enter it (or Home).
109. Right-click in empty space > Create > Folder. Name: footballiq.
110. Inside footballiq, create 4 sub-folders:

Folder	Purpose
00_setup	One notebook: mount ADLS and verify connection
01_bronze	12 notebooks: one per CSV file
02_silver	10 notebooks: dim tables + fact tables + quarantine
03_gold	4 notebooks: business KPI tables

STEP 10.2

Read ADLS Files Using External Location Path

5 min

Since you are using External Locations (Section 8) instead of the old mount approach, the path to read from ADLS is the ABFS path directly — no mounting needed:

```
# Read CSV using the External Location ABFS path directly
# No mounting required — External Location handles the auth
raw_path = 'abfss://raw@adlsfootballiqdev.dfs.core.windows.net/appearances/appearances.csv'

df = spark.read.option('header','true').option('inferSchema','false').csv(raw_path)
df.show(5)
```

If you prefer the old `dbutils.fs.mount()` approach

The old mount approach still works in 2025 but is considered legacy.

If you want to use mounts, fetch the key from Key Vault and mount with `wasbs://` protocol.

For this portfolio, use External Location (`abfss://` path directly) — it is the current best practice and shows interviewers you know the modern Unity Catalog approach.

STEP 10.3

Bronze Ingestion Template Notebook

15 min

Guthub :

https://github.com/saizdevlab/de-project-footballiq-azure-resource-group/blob/Dev/01_bronze/01_bronze_all_files_data_ingest.ipynb.ipynb

SOURCE_PATH folder	SOURCE_FILE	TARGET_TABLE
club_games/	club_games.csv	football_catalog.bronze.club_games
clubs/	clubs.csv	football_catalog.bronze.clubs
competitions/	competitions.csv	football_catalog.bronze.competitions
countries/	countries.csv	football_catalog.bronze.countries
game_events/	game_events.csv	football_catalog.bronze.game_events
game_lineups/	game_lineups.csv	football_catalog.bronze.game_lineups
games/	games.csv	football_catalog.bronze.games
national_teams/	national_teams.csv	football_catalog.bronze.national_teams
player_valuations/	player_valuations.csv	football_catalog.bronze.player_valuations
players/	players.csv	football_catalog.bronze.players
transfers/	transfers.csv	football_catalog.bronze.transfers

After, verify in Catalog Explorer: football_catalog > bronze should show 12 tables.

11. Silver Layer — Validate, Type & Build Dimensional Model

Silver reads from Bronze, enforces types, fixes quality issues, validates foreign keys, and builds proper star schema tables. Bad rows go to a quarantine table with a reason.

Silver Layer Rules

1. Read ONLY from football_catalog.bronze — never from ADLS directly.
2. Cast all columns from string to correct types (dates, integers, floats).
3. Handle nulls: fill with defaults, drop if critical FK is null, or quarantine with reason.
4. Implement SCD Type 2 on dim_clubs and dim_players (tracks when players change clubs).
5. Validate FK: every player_id in fact_appearances must exist in dim_players.

6. Bad rows (FK violations, critical nulls) go to football_catalog.silver.quarantine.
7. Use MERGE INTO (upsert) on fact tables so re-running the pipeline is safe.

Github :

https://github.com/saizdevlab/de-project-footballiq-azure-resource-group/tree/Dev/02_silver

12. Gold Layer — Business-Ready Aggregations

Gold reads only from Silver. These tables are what analysts and Power BI connect to. Use window functions for rankings.

12.1 Gold:

Github :

https://github.com/saizdevlab/de-project-footballiq-azure-resource-group/tree/Dev/03_gold

13. Orchestrate with Databricks Workflows

Databricks Workflows (built-in job scheduler) runs all your notebooks in sequence automatically. No Azure Data Factory needed — everything stays inside Databricks.

STEP 13 Create the footballiq_pipeline Workflow

15 min

111. In the Databricks left sidebar, click Workflows (the clock/workflow icon).

112. Click + Create job (top right).

113. Name the job: footballiq_pipeline.

114. You see a task graph canvas. Create 3 tasks:

Task Name	Notebooks	Depends On
bronze_ingest	All 12 notebooks in 01_bronze folder	None (runs first)

silver_transform	All notebooks in 02_silver folder	bronze_ingest
gold_aggregate	All notebooks in 03_gold folder	silver_transform

115. For each task, click + Add task. Set Type: Notebook. Browse to the folder.
116. Set Compute: Serverless for each task (or footballiq-cluster if you have a classic cluster).
117. For silver_transform: in the Depends on field, select bronze_ingest.
118. For gold_aggregate: in the Depends on field, select silver_transform.
119. Click Add notification under the job settings > add your email > select On failure.
120. Click Run now to test the full end-to-end pipeline.
121. Click the Runs tab to monitor — each task shows Success (green) or Failed (red).

What good orchestration looks like in an interview

Bronze task = 12 notebooks run in parallel (same task, multiple notebook paths).

Silver task waits for Bronze to fully succeed before starting.

Gold task only runs if Silver succeeds.

On failure, you get an email notification automatically.

This is exactly how production DE teams run daily/weekly pipelines.

14. Connect Power BI to Gold Layer

STEP 14.1 Create a SQL Warehouse

5 min

122. In Databricks, click SQL Warehouses in the left sidebar.
123. Click + Create SQL Warehouse.
124. Name: footballiq-sql. Size: 2X-Small (cheapest). Click Create.
125. Once running, click on the warehouse name > Connection details tab.
126. Copy: Server hostname and HTTP path — you need these for Power BI.

STEP 14.2 Connect Power BI Desktop

5 min

127. Open Power BI Desktop.
128. Click Get data > search for Azure Databricks > Connect.
129. Paste Server hostname and HTTP path. Click OK.
130. For authentication, select Microsoft Account and sign in with your Azure account.
131. In the Navigator, expand football_catalog > gold.
132. Select player_season_stats, club_league_table, top_transfers. Click Load.
133. Build 3 report pages: League Table, Top Scorers, Transfer Market.

15. Final Checklist — Portfolio Readiness

☐	Checklist Item
☐	Resource group rg-footballiq-dev created with all resources in same region
☐	ADLS Gen2 with hierarchical namespace enabled + 12 raw folders + all CSVs uploaded
☐	Key Vault: Vault access policy selected + AzureDatabricks principal (2ff814a6...) has Get+List
☐	ADLS access key stored in Key Vault as 'adls-footballiq-key'
☐	Databricks PREMIUM workspace created (not Standard — required for Unity Catalog)
☐	Compute ready: Serverless attached in a test notebook (or classic cluster if your workspace has it)
☐	Unity Catalog confirmed active (Catalog Explorer shows catalogs)
☐	Access Connector created + Storage Blob Data Contributor role on ADLS
☐	Storage Credential and External Location created in Databricks Catalog Explorer
☐	Secret Scope footballiq-kv-scope created via #secrets/createScope URL
☐	Secret scope tested: dbutils.secrets.get() returns [REDACTED]
☐	football_catalog created with bronze, silver, gold schemas
☐	All 12 Bronze notebooks run — 12 Delta tables in football_catalog.bronze
☐	Silver dim tables: dim_competitions, dim_countries, dim_clubs (SCD2), dim_players (SCD2)
☐	Silver fact tables: fact_games, fact_appearances with FK validation
☐	Quarantine table populated with bad rows + reason column
☐	Gold tables: player_season_stats, club_league_table, top_transfers, match_summary
☐	Databricks Workflow (3 tasks with dependencies + email on failure) runs end-to-end

☐	Power BI Desktop connected to Gold layer via SQL Warehouse
☐	GitHub repo with README, architecture diagram, and notebook screenshots

Interview talking points for this project

"I built an end-to-end pipeline on Azure using the medallion architecture with Unity Catalog."

"All 12 Transfermarkt source CSVs land in an ADLS Gen2 raw container via an External Location."

"Bronze ingests raw data as Delta tables with metadata columns. Silver implements SCD Type 2 on dim_players and dim_clubs, FK validation on fact tables, and routes bad rows to a quarantine table."

"Gold produces analyst-ready aggregates: player season stats with window-function rankings, league

standings, and transfer market summaries."

"The pipeline is orchestrated by a Databricks Workflow with 3 dependent tasks and email alerting."

"Power BI connects to the Gold layer via Databricks SQL Warehouse using DirectQuery."